Fundamentals of Deep Learning(Assignment-1)

Q1. Compare Deep Learning and Shallow Machine Learning using real datasets.

Task:

1. Select any dataset (Iris, MNIST subset, or any CSV dataset).
2. Train a shallow ML model (e.g., logistic regression, decision tree).
3. Train a simple deep neural network on the same dataset.

Compare:

Accuracy

Training time

Overfitting behavior

Feature engineering requirement

Deliverables:

Code

Accuracy comparison table

5–7 line conclusion

Q.2 Demonstrate Gradient Descent and Batch Optimization.

Task:

Implement **Gradient Descent**, **Stochastic Gradient Descent**, and **Mini-Batch Gradient Descent** for a simple regression task.

Compare convergence speed and stability.
Deliverables:

Code

Graphs of all optimization techniques

Short analysis (6–8 lines)

Q.3 Simulate a Biological Neuron vs. Artificial Neuron.

Task:

Draw a diagram (handmade or digital) comparing biological neuron components with artificial neuron components.

Write Python code to implement an artificial neuron with:

Weighted sum

Activation function (ReLU/Sigmoid)

Deliverables:

Diagram

Code + sample input/output

Explanation of similarities/differences

Build a Single-Layer Perceptron for a Binary Classification Task.

Task:

Train a perceptron on a linearly separable dataset.

Plot the learned decision boundary.

Deliverables:

Code

Decision boundary figure

Summary of training process

Fundamentals of Deep Learning(Assignment-1)

Q1. Compare Deep Learning and Shallow Machine Learning using real datasets.

➔ Shallow Machine Learning (Decision Tree)

CODE:

```
[22] ✓ 0s
import numpy as np
import time
from tensorflow.keras.datasets import mnist
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

[23] ✓ 0s
(X_train, y_train), (X_test, y_test) = mnist.load_data()
X_train = X_train/255.0
X_test = X_test/255.0

[24] ✓ 0s
X_train_flat = X_train.reshape(X_train.shape[0],-1)
X_test_flat = X_test.reshape(X_test.shape[0],-1)

[21] ✓ 17s
start = time.time()
shallow_model = DecisionTreeClassifier(max_depth=20, random_state=42)
shallow_model.fit(X_train_flat, y_train)
shallow_model_time = time.time()-start

y_pred_shallow = shallow_model.predict(X_test_flat)
shallow_accuracy = accuracy_score(y_test, y_pred_shallow)

[25] ✓ 0s
print("Decision Tree Accuracy:", shallow_accuracy)
print("Decision Tree Training Time:", shallow_model_time)

Decision Tree Accuracy: 0.8818
Decision Tree Training Time: 17.795822381973267
```

Overfitting and Feature Selection Overview :

The model demonstrates clear overfitting because a deep decision tree (max_depth = 20) is trained on raw MNIST pixel data containing 784 input features per image. This high model complexity allows the tree to learn very detailed and dataset-specific patterns, including noise, rather than learning generalizable digit representations. As a result, the model tends to memorize training samples instead of capturing meaningful structures, which limits its ability to generalize to unseen test data. This behavior highlights the importance of feature selection, since many pixel features are redundant, highly correlated, or irrelevant (e.g., background pixels). Retaining all features increases model complexity and overfitting risk, whereas selecting or reducing features would improve generalization, reduce noise, and lead to more stable and efficient learning performance.

➔ Deep Learning

CODE:

```
[31]
import numpy as np
import time
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.utils import to_categorical
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

[32]
(X_train, y_train), (X_test, y_test) = mnist.load_data()
X_train = X_train/255.0
X_test = X_test/255.0

[33]
X_train_flat = X_train.reshape(X_train.shape[0],-1)
X_test_flat = X_test.reshape(X_test.shape[0],-1)
y_train_cat = to_categorical(y_train)
y_test_cat = to_categorical(y_test)

[34]
start = time.time()
dnn = Sequential([
    Dense(128, activation='relu', input_shape=(784,)),
    Dense(64, activation='relu'),
    Dense(10, activation='softmax')
])
dnn.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
dnn.fit(X_train_flat, y_train_cat, epochs=10, batch_size=128, verbose=1)
dnn_train_time = time.time() - start

loss, dnn_acc = dnn.evaluate(X_test_flat, y_test_cat, verbose=0)
Show hidden output

[35]
print("Deep Neural Network Accuracy:", dnn_acc)
print("Deep Learning Training Time:", dnn_train_time)

Deep Neural Network Accuracy: 0.9789999723434448
Deep Learning Training Time: 27.49014973640442
```

Overfitting and Feature Selection Overview:

This deep neural network shows much better generalization behavior compared to the decision tree because it learns hierarchical feature representations from the raw pixel data instead of relying on direct pixel-level splits. The multi-layer structure with nonlinear activations (ReLU) enables the model to automatically extract meaningful patterns such as edges, shapes, and digit structures, which allows it to generalize well to unseen data, as reflected by the high test accuracy. Although the model still uses all 784 pixel features, it does not suffer from severe overfitting because the learning process focuses on shared patterns across samples rather than memorization. In this case, explicit feature selection is less critical, since the neural network performs implicit feature learning through its hidden layers. However, dimensionality reduction or feature engineering could still improve efficiency and training time, even though generalization performance is already strong.

CONCLUSION:

Overall, the two models demonstrate clear differences in learning behaviour and generalization performance. The **decision tree model** exhibits strong overfitting due to its high depth and training on high-dimensional raw pixel data, which leads it to memorize noise and dataset-specific patterns rather than learning generalizable digit representations. This results in limited generalization performance and highlights the importance of controlling model complexity and applying feature selection or dimensionality reduction. In contrast, the **deep neural network model** achieves significantly higher accuracy by learning hierarchical feature representations directly from the data, enabling it to extract meaningful structures such as edges, shapes, and digit patterns. This allows the neural network to generalize effectively to unseen data without explicit feature selection, as feature learning is performed implicitly through its layered architecture. Together, these results show that model choice and data representation play a critical role in performance: simple models with high-dimensional raw features are prone to overfitting, while deep learning models are better suited for complex data such as images due to their ability to learn robust and generalizable features automatically.

Q.2 Demonstrate Gradient Descent and Batch Optimization.

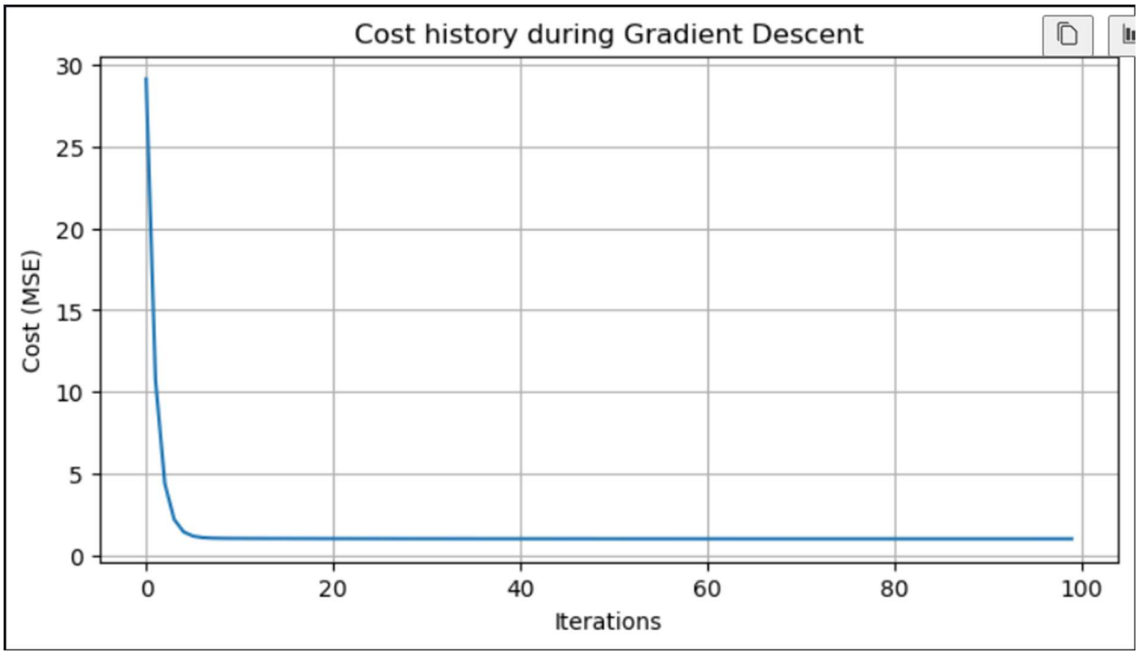
```python
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(0)

X = 2 * np.random.rand(100, 1)
y = 4 + 3 * X + np.random.randn(100, 1)

X_b = np.c_[np.ones((100, 1)), X]
```

✓ 0.0s Python

-> Gradient Descent

```python
def batch_gd(X, y, lr=0.1, epochs=100):

    theta = np.random.randn(2, 1)
    cost = []

    m = len(X)

    for i in range(epochs):
        gradients = (2/m) * X.T.dot(X.dot(theta) - y)
        theta = theta - lr * gradients

        predictions = X.dot(theta)
        cost.append(np.mean((predictions - y) ** 2))

    return cost

batch_losses = batch_gd(X_b, y)

plt.figure(figsize=(8,4))
plt.plot(range(100), batch_losses)
plt.xlabel("Iterations")
plt.ylabel("Cost (MSE)")
plt.title(" Gradient Descent")
plt.grid(True)
plt.show()
```

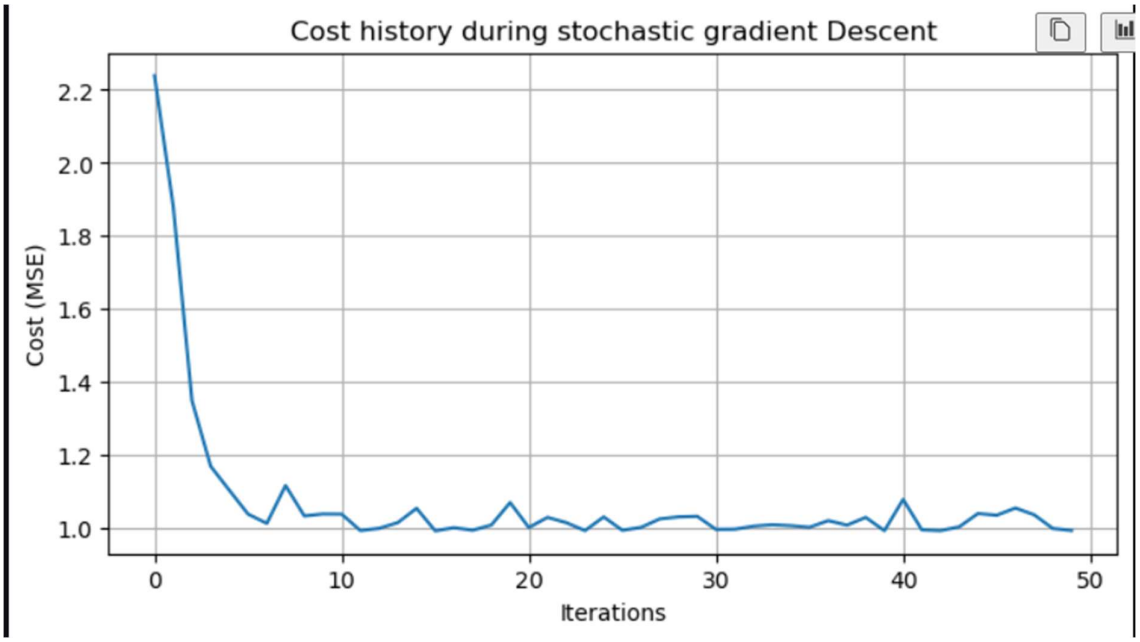
✓ 0.0s Python



-> Stochastic Gradient Descent

```python
def sgd(X, y, lr=0.01, epochs=50):

    theta = np.random.randn(2, 1)
    cost = []

    m = len(X)

    for epoch in range(epochs):

        for i in range(m):

            random_index = np.random.randint(m)

            xi = X[random_index:random_index+1]
            yi = y[random_index:random_index+1]

            gradients = 2 * xi.T.dot(xi.dot(theta) - yi)
            theta = theta - lr * gradients

        predictions = X.dot(theta)
        cost.append(np.mean((predictions - y) ** 2))

    return cost

Sgd_losses = sgd(X_b, y)

plt.figure(figsize=(8,4))
plt.plot(range(50), Sgd_losses)
plt.xlabel("Iterations")
plt.ylabel("Cost (MSE)")
plt.title("stochastic gradient Descent")
plt.grid(True)
plt.show()
```

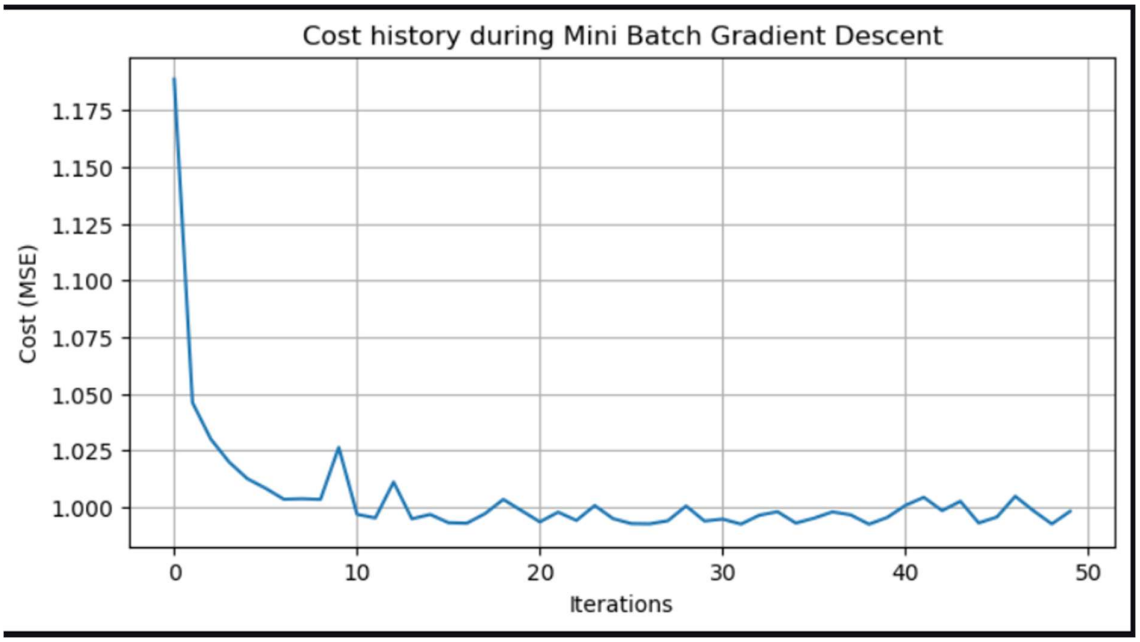
✓ 0.1s



-> Mini Batch Gradient Descent

```python
def mini_batch_gd(X, y, lr=0.1, epochs=50, batch_size=20):

    theta = np.random.randn(2, 1)
    cost = []

    m = len(X)

    for epoch in range(epochs):

        indices = np.random.permutation(m)
        X_shuffled = X[indices]
        y_shuffled = y[indices]

        for i in range(0, m, batch_size):

            xi = X_shuffled[i:i+batch_size]
            yi = y_shuffled[i:i+batch_size]

            gradients = (2/len(xi)) * xi.T.dot(xi.dot(theta) - yi)
            theta = theta - lr * gradients

        predictions = X.dot(theta)
        cost.append(np.mean((predictions - y) ** 2))

    return cost

mini_batch_gd_losses = mini_batch_gd(X_b, y)

plt.figure(figsize=(8,4))
plt.plot(range(50), mini_batch_gd_losses)
plt.xlabel("Iterations")
plt.ylabel("Cost (MSE)")
plt.title("Mini Batch Gradient Descent")
plt.grid(True)
plt.show()
```

✓ 0.0s Python



Conclusion:

The three gradient descent methods represent different strategies for minimizing a cost function. Batch Gradient Descent updates the model using the entire dataset at each step, which produces stable and smooth convergence but results in slower updates and higher computation per iteration. Stochastic Gradient Descent updates the model using only one randomly selected data point at a time, which leads to faster updates but introduces high variability and fluctuations in the cost function, producing a noisy and uneven convergence curve. Mini-Batch Gradient Descent provides a balanced approach by using small subsets of data for each update, achieving a good compromise between speed and stability. It typically converges faster than SGD with smoother behavior, while being more efficient than BGD, making it the most practical and scalable choice for large datasets.